

Lexora: 書記を一次とする言語開発試行

Kammultica

概要

本稿は、音声チャンネルに最適化されてきた既存の自然言語が、空間配置や並列提示を活用できる書記モードに対して必ずしも最適ではないという問題意識から出発し、書記を一次対象とする言語表現層 Lexora を提案する。Lexora は、あらゆる自然言語の語彙を原形のままノード (Lexon) として受け容れつつ、時制・相・法・態・極性・数量・定性・情報構造・証拠性・意味役割・時空間・比較などの文法範疇を語形から切り離し、型付き属性として明示的に付与する多次元構造を採用する。これにより、従来は語順や屈折に埋め込まれていた統語・談話情報を、構造関係と属性束の組み合わせとして宣言的に表現し、高精度かつ低コンテキストで機械可読・人間可解な記述を可能にすることを目指す。さらに、Lexon と属性束からなる擬似コードレベルの仕様を提示し、翻訳中間表現、技術文書、仕様書、法令など、文脈依存性の低減が重要となる領域への応用可能性を議論する。

1 導入

本稿の目的は、「書記 (writing) を一次対象とする言語表現」を設計射程として定義し、その理論的動機・実装原理を論じた上で、その具体化である構造化言語表現形式『Lexora』を提示することにある。

一般に、自然言語は聴覚・音声チャンネルを主とする相互行為の中で発達してきたと考えられており、書記は歴史的には後発の技術として定着してきた (Hockett 1960 [1])。書記の普及は教育制度や媒体アクセスに依存し、世界では依然 7.39 億人規模、割合にして約 9% の非識字人口が存在する (UNESCO 2025 [2])。こうした観点から、人類言語の標準的な使用様式は歴史的に「まず音声」であり、書記はその後に補助的モードとして成立・制度化されてきたと言える。

しかし、音声モードを前提として形成された言語構造は、必ずしも書記モードの要請と一致しない。例えば、多くの自然言語にみられる屈折 (inflection)・一致 (agreement)・格標識などは、文中の統語的關係 (人称・数・時制・法・格など) を語形に埋め込むことで、線形的かつ同時性の制約が強い音声チャンネルでの識別性・冗長性・処理容易性を確保する複合的装置として発達してきたと考えられる (Booij 2007 [3])。

ところが、現代の情報環境では、書記 (視覚) チャンネルを前提とする情報設計・流通の比重が飛躍的に増大している。ヒトの処理速度という観点でも、成人の黙読速度は平均で 238-260 語/分程度 (英語・素材依存) と見積もられ、聴取・発話の「快適速度」150 語/分前後を一般に上回る (メタ分析および実務的ガイドの整合的報告) (Brysbaert 2019 [4])。加えて、計算機側の処理能力も既に一般的な家庭用ラップトップの水準でヒトの読解速度を桁違いに上回っている。Chen and Manning (Chen and Manning Oct. 2014 [5]) の遷移ベース依存構文解析器では、Intel Core i7 2.7 GHz/16 GB RAM という汎用的な CPU 環境で、Penn Treebank 英語に対して 92% 超の解析精度を維持しながら 1000 文/秒以上を処理できる。

さらに音声では、言語に依存しつつも平均情報率がほぼ一定の帯域に制約される一方、書記ではレイアウト・参照・非線形構造・グラフィック等の多重チャンネルを空間的に同時提示できるため、意味設計面の同時並行性をはるかに拡張できる (Coupé et al. 2019 [6])。この「同時並行的・低コンテキスト」提示能力は、音声モードを前提に組まれた屈折的機構や語順上の手続き的合図を、より直接的な標識 (時刻・場所・因果・根拠・情報源などのメタデータ化) へと再設計する契機を与える。

以上の、書記の音声に対する処理スループットについての優位性を踏まえて、本稿は「書記を本義とする言語 (writing-oriented language)」の構築を提案する。その主眼は以下の三点に要約できる。

- 書記の並列提示と参照の即地化を活かし、高精度 (high-precision)・低コンテキスト (explicit, self-contained) に構造化された記述を実現する。
- 全言語の語彙を横断的に取り込み、母語レキシコンとシームレスに接続することで、書記上の統一インターフェースを提供する。活用依存の処理はオブジェクトの属性へと移送し、原形 (辞書形) + 属性の単純化された変換を可能にする。
- 目に映る光景や自身の思考を表現するといった自然言語の本来的な目的を果たせる、哲学言語的に本質的な最小属性集合を型理論的に定義して概念上不可欠な最小属性群に還元し、不要な語形的冗長性を排するコンパクトな表現を追求する。

本稿の位置づけとして、文法・統語機能の共通化に関する先行は以下の三系統に整理できる。

- 言語横断アノテーション規約: 品詞・形態・依存関係を普遍タグ体系と設計指針で統一し、既存ツリーバンクをその共通スキームへ写像する (UPOS/UniMorph の普遍タグ + UD の依存ラベル設計とガイドライン) (Petrov, Das, and McDonald 2012; Kirov et al. 2018; Nivre and al. 2020)
- 抽象構文を共有する文法工学: 言語非依存の抽象構文 (型付き範疇・構成子) を一元的に定義し、各言語の具体構文へは線形化規則や言語別パラメータで写像する二層構成 (GF), あるいは理論文法 (HPSG/LFG) の言語横断コアに現象別モジュールを差し込む生成方式 (Grammar Matrix/ParGram) (Ranta 2004; Bender et al. 2010; Butt et al. 2002)
- 言語非依存の意味表現: 表層差を捨象して命題・役割関係をグラフ/制約として記述する設計で、概念ノード + 役割辺の有向ラベル付きグラフ (AMR), Scene と参加者を中核とする DAG (UCCA), 未特定化を保持する平坦論理形式 (MRS) という API 様式 (Banarescu et al. 2013; Abend and Rappoport 2013; Copestake et al. 2005)

本稿が提案する Lexora は、それぞれの系譜と親和的でありつつも、「語形」ではなく「属性 (メタデータ)」を一次的な記号単位とみなす点において一段と書記指向である。音声モードが必要とする判別の「手続き」を、書記モードにおける明示的標識へと移送することで、機械可読性・人間可解性・翻訳可能性の同時最適化を狙い、翻訳中間表現・技術文書・仕様・法律など、低コンテキスト化が価値を持つ領域において効果を発揮することを目指す。

総じて、本稿は以下を主張する。歴史的・機能的に音声に最適化されてきた自然言語は、書記というモードの情報設計に対して必ずしも最適ではない。読み手がアクセスし得る視覚的・空間的・非線形的資源を最大化するためには、屈折や語順などの時間展開に依存する合図を、構造・属性・リンクとして再符号化するほうが、より低コンテキストで高精度な記述をもたらす。本稿の後続節では、この設計思想を具体化した Lexora の仕様と実装 (擬似コード) を提示し、その応用可能性を検討する。

2 設計

2.1 設計理念

Lexora が目指すのは、従来の音声言語の線形性に強く拘束された書記から離脱し、記述を一次元の並びではなく多次元的な構造として表現できるようにすることである。

これまでの書記は、音声言語の運動をなぞる形で一列の線形的な文字列として設計・構成されてきたが、Lexora では書記を独立した情報チャネルとして再定義し、節・句・語の支配関係や修飾関係などを明示的に構造化することで多次元化を実現する。そのために Lexora は、すべての自然言語を対象とする汎用的な表現層として、文字によって記述可能なあらゆる自然言語の語彙を原形のまま横断的に取り込み、従来は語形変化に埋め込まれていた法 (mood)・相 (aspect)・態 (voice) や時制 (tense) などの情報を、独立した「属性」として保持する「オブジェクト」を最小要素に据える。

Lexora とは、バベルの塔の再建計画である。すなわち、いかなる自然言語からも機械的に転写可能でありつつ、いずれの自然言語にも偏らない中立的かつ普遍的な表現層として機能し、「世界や思考を記述する」ために必要な機能一式を備えた完全言語を志向するものである。Lexora を媒介として用いることで、任意の自然言語を別の自然言語への相互翻訳が可能となるだけでなく、より長期的には、現在のように言語圏ごとに異なる自然言語を運用する状況から脱却し、全人類が共有する唯一の言語コードとして採択されることを展望している。

また、Lexora の解釈主体は人間に限定されない。Lexora 形式による記述は人間が直接視覚的に読み取ることでも可能ではあるが、従来の書記表現に比して情報量と文字数が著しく増大するため、そのままでは煩瑣になりやすい。本稿では、Lexora 形式をネイティブに処理する計算機システムが一次解釈を担い、その結果を人間にとって認知的負荷の低い空間配置・可視化形式へと再構成することを前提とする。この設計方針は、将来的な脳-機械インタフェース (BMI) やマインドアップロード等による認知能力の拡張をも視野に入れたものであり、最終的には計算機による仲介的な一次解釈を不要とし、多次元的に並列化されたチャネルを介した高速かつダイレクトな理解が実現される可能性を見込む。

2.2 設計方針

2.2.1 動作環境

各種プログラミング言語により動作する想定のもと、本稿では擬似コードで実装する。

2.2.2 機能要件

Lexora の具体的な設計をおこなうにあたり、具体的な柱とするのは以下の 3 つである。

オブジェクト「Lexon」を定義ノードのラベルとして、各自然言語の全語彙の原形を使用する。

- ・「語」から、多数の属性を具えた「オブジェクト」へ: 音声起源の語形への情報埋め込みを廃止し、原形+属性のオブジェクトを最小単位とする。
- ・「語順」から、「ポイントによる連結」へ: 習慣的な性格が大きい語順ではなく、オブジェクトの属性へのポイントの連結で文構造を明示化する。
- ・学習コスト・推論負荷の極小化: 意味をより限定的に精確に表現できるフォーマットを提供することで、学習コストや推論負荷の削減に努める。

2.2.3 コアオブジェクト: Lexon

Lexora はオブジェクト「Lexon」を最小単位とし、これの多次元的な連結で文を構成する。

Lexon は、*null* 許容の語彙形 *text* を持つ抽象ノードであり、統語-意味-談話の結節点として振る舞う。主要フィールドを以下に要約する。

- ・構造関係: *dominatee* (主要部-被支配), *modiffee* (修飾-被修飾), *coordination* (並列関係)
- ・時間/空間: *spatiotemporalKind* (時間性・空間性), *direction (from/to)* (方向の始点・終点), *time* (時刻), *place* (場所), *frequency* (頻度), *intensity* (強度)
- ・数量/指示: *number* (文法数+実数値), *quantType* (量化 全称/单称), *definiteness* (定性)
- ・意味役割: *roles* (格 ACTOR/UNDERGOER/EXPERIENCER/STIMULUS/RECIPIENT/CONTENT/COMITATIVE/POSSESSOR/STANDARD/DOMAIN), *instrument* (道具)
- ・法/様相/態: *negativePolarity* (否定), *mood* (法), *modality* (様相), *passiveVoice* (受動態), *causativeVoice* (使役態), *middleVoice* (中動態), *reflexive* (帰態), *reciprocal* (相互態)
- ・構文: *negativePolarity* (否定), *register* (敬語 POLITE/HONORIFIC/HUMBLE)
- ・談話/根拠/比較: *infoStructure (TOPIC/FOCUS)*, *evidentiality* (INFERENCE/ASSUMPTION/HEARSAY/QUOTATIVE), *degreeKind* (比較級/最上級/等級)

3 実装

3.1 基底オブジェクト: Lexon

まず、自然言語における語 (word) にあたるオブジェクトと、そのコンストラクタ関数を作成する。

普遍的な言語表現として、語 word ではない、語・句 (複数語の set)・節 (主語述語を含む句)・文 (1 つ以上の節から成り、他の文との境界を持つもの) すべてを表現しうる新たな基礎単位を用意する。本稿ではこの基礎単位を *lexis* + *-on* から **Lexon(言素)**、これを基礎単位とする表現体系を **Lexora(言素言語)** と、それぞれ命名する。

なお、関係性のみを示す場合など、Lexon 間の接着に用いる場合は *text* を *null* としたものを扱うことができる。

```
1 Class Lexon
2   Attributes:
3     // lexical text, or null for abstract relation nodes
4     text: string | null
5
6   Constructor(inputText: string | null):
7     text = inputText
```

Listing 1 クラス定義

これを用いて、以下のように任意の文字列をコンストラクタ関数の引数 *text* に渡すことで、Lexora 空間内にインスタンスを生成することができる。

```
1 // Example: Create an instance of Lexon with text "Hello"
2 lex1 = new Lexon("Hello")
3
4 // Example: Create another instance of Lexon with text "el elemento"
5 lex2 = new Lexon("el elemento")
```

Listing 2 Lexon インスタンス化

本稿で実装する完全言語は、このオブジェクト Lexon に幾つかの本質的な概念を属性として与え、任意の数のインスタンスをポインタ参照で繋ぐことで自由に文を作成できるようにするものである。

本稿で定義されるオブジェクトはこの一つだけである。よって以後、Lexon は単に**オブジェクト**と呼ぶ。

基本的に、コンストラクタに渡す引数 *inputText* は、原形 (辞書形) の語のみである。当該言語において動詞または名詞が存在しない語については、それを渡してもよい。あるいはまた、話者間において了解さえ取れば、どんな記号・文字・画像などのデータ、を渡してもよい。

3.2 コア属性

本節では、Lexon の核となる属性を実装する。「世界や思考を記述する」という自然言語本来の原点に立ち返り、根源的な目的を達成するために最低限必要なもの、という位置づけである。

3.2.1 支配

発話の単位を言語学に則って文 (sentence) と呼ぶとき、Lexora において文は 1 つ以上のオブジェクトから成る。言語学において、主語と述語を一つずつ含む集まりは節 (clause) と呼ばれ、挨拶や定型句のみの文を除けば、たいていの文は 1 つ以上の節から成る。この標準的な形を表現するために、オブジェクトの属性を 1 つと、その setter 関数 1 つを追加する。

```
1 Attributes:
```

```

2  dominatee: Lexon*
3
4  Method SetDominatee(dominateeObj: Lexon*):
5      dominatee = dominateeObj->dominatee

```

Listing 3 支配属性追加

ここで *dominatee* は、当該オブジェクトが支配する対象のことである。主語述語対応や主語補語対応などを、支配者・被支配者の関係として抽象化する。オブジェクトに品詞の別はないが、例えばあるデコンパイラは品詞に応じて、以下のように解釈することができる。

オブジェクトが動詞を支配するとき、このオブジェクトは ****SV**** の動詞句となる。

```

1  // Create subject (S) "I"
2  S = new Lexon("I")
3
4  // Create verb (V) "go"
5  V = new Lexon("go")
6
7  // Subject (S) dominates verb (V), forming an SV phrase
8  S.SetDominatee(V)

```

Listing 4 SV 支配

オブジェクトが名詞または形容詞を支配するとき、このオブジェクトは繫辞 (copular) 的に両者を結びつけることができる。

```

1  // Create noun (N1) "Socrates"
2  N1 = new Lexon("Socrates")
3
4  // Create noun (N2) "man"
5  N2 = new Lexon("man")
6
7  // N1 dominates N2 → "Socrates is man"
8  N1.SetDominatee(N2)

```

Listing 5 名詞 A が名詞 B を支配する場合

```

1  // Create noun (N) "sky"
2  N = new Lexon("sky")
3
4  // Create adjective (Adj) "blue"
5  Adj = new Lexon("blue")
6
7  // N dominates Adj → "Sky is blue"
8  N.SetDominatee(Adj)

```

Listing 6 名詞 A が形容詞 B を支配する場合

応用として、以下の構造を表現することができる。

```

1  // Create subject (S) "I"
2  S = new Lexon("I")
3
4  // Create verb (V) "see"
5  V = new Lexon("see")
6
7  // Create object (O) "you"
8  O = new Lexon("you")

```

```

9
10 // S dominates V → SV phrase
11 S.SetDominatee(V)
12
13 // V dominates O → VO phrase
14 V.SetDominatee(O)

```

Listing 7 SVO 構造

```

1 // Create subject (S) "I"
2 S = new Lexon("I")
3
4 // Create verb (V) "consider"
5 V = new Lexon("consider")
6
7 // Create object (O) "you"
8 O = new Lexon("you")
9
10 // Create complement (C) "smart"
11 C = new Lexon("smart")
12
13 // S dominates V → SV phrase
14 S.SetDominatee(V)
15
16 // V dominates O → VO phrase
17 V.SetDominatee(O)
18
19 // O dominates C → OC phrase
20 O.SetDominatee(C)

```

Listing 8 SVOC 構造

3.2.2 修飾

一般に、形容詞は名詞を、副詞は動詞を、それぞれ修飾する。言い換えれば、修飾対象によってその修飾者は形容詞・副詞のいずれかに分類されるが、これは語形標識のために必要なものであり、修飾者 (modifier) と被修飾者 (modiffee) が明示されてさえいれば、この分類は不要となる。この標準的な形を表現するために、自身の修飾対象を示すオブジェクトの属性を 1 つと、その setter 関数 1 つを追加する。

```

1 Attributes:
2   modiffee: Lexon*
3
4 Method setModiffee(modiffeeObj: Lexon*):
5   modiffee = Obj->modiffeeObj

```

Listing 9 修飾対象属性追加

これにより、形容詞・副詞による修飾は次のような形で一本化されていることがわかる。

```

1 // Create adjective (Modifier) "red"
2 Modifier = new Lexon("red")
3
4 // Create noun (Modiffee) "apple"
5 Modiffee = new Lexon("apple")
6
7 // Modifier "red" modifies noun "apple"
8 Modifier.setModiffee(Modiffee)

```

Listing 10 形容詞による名詞修飾

```

1 // Create adverb (Modifier) "quickly"
2 Modifier = new Lexon("quickly")
3
4 // Create verb (Modiffee) "run"
5 Modiffee = new Lexon("run")
6
7 // Modifier "quickly" modifies verb "run"
8 Modifier.setModiffee(Modiffee)

```

Listing 11 形容動詞による動詞修飾

3.2.3 並列

対象が複数存在する場合に並列に並べることもまた、essential である。

それぞれの並列表現、等位接続 (AND)・代替的接続 (OR)・逆接的並列 (BUT)・同格的接続 (APPOSITIVE), のいずれであるかを標識するための Enum ならびにサブクラスをまず定義する。

```

1 Enum CoordinationType:
2     AND // conjunctive coordination ("and", simple listing)
3     OR // disjunctive coordination ("or", alternatives)
4     BUT // adversative coordination ("but", contrast)
5     APPOSITIVE // appositive pairing ("John, teacher")
6
7 Class Coordination
8     Attributes:
9         type: CoordinationType // logical/semantic type of coordination
10        members: Lexon*[] // coordinated elements (2 or more)
11
12    Constructor(t: CoordinationType, xs: Lexon*[]):
13        type = t
14        members = xs
15
16    Method setType(t: CoordinationType): type = t
17    Method setMembers(xs: Lexon*[]): members = xs
18    Method addMember(x: Lexon*): members.append(x)

```

Listing 12 並列サブクラス定義

そしてこのサブクラスを属性とし、その setter 関数と共にオブジェクトへ追加する。

```

1 Attributes:
2     coordination: Coordination* | null // coordination metadata for this node
3
4 Method setCoordination(c: Coordination*):
5     coordination = c

```

Listing 13 並列属性追加

```

1 // Example: Noun coordination — "apple and orange"
2 A = new Lexon("apple")
3 B = new Lexon("orange")
4
5 Group = new Coordination(CoordinationType.AND, [A, B])
6

```

```

7 Node = new Lexon(null) // abstract coordination node
8 Node.setCoordination(Group) // holds the coordination structure

```

Listing 14 等位接続

```

1 // Example: Verb coordination — "run or jump"
2 V1 = new Lexon("run")
3 V2 = new Lexon("jump")
4
5 Group = new Coordination(CoordinationType.OR, [V1, V2])
6 Node = new Lexon(null)
7 Node.setCoordination(Group)

```

Listing 15 代替的接続

```

1 // Example: Adversative adjectives — "tall but weak"
2 Adj1 = new Lexon("tall")
3 Adj2 = new Lexon("weak")
4
5 Group = new Coordination(CoordinationType.BUT, [Adj1, Adj2])
6 Node = new Lexon(null)
7 Node.setCoordination(Group)

```

Listing 16 逆説的並列

```

1 // Example: Appositive — "John, teacher"
2 N1 = new Lexon("John")
3 N2 = new Lexon("teacher")
4
5 Group = new Coordination(CoordinationType.APPOSITIVE, [N1, N2])
6 Node = new Lexon(null)
7 Node.setCoordination(Group)

```

Listing 17 同格的並列

```

1 // Example: Three-way conjunction — "A, B, and C"
2 A = new Lexon("A")
3 B = new Lexon("B")
4 C = new Lexon("C")
5
6 Group = new Coordination(CoordinationType.AND, [A, B, C])
7 Node = new Lexon(null)
8 Node.setCoordination(Group)

```

Listing 18 三項並列

3.2.4 数性

同質と見做す任意の対象・対象群については、その数性を標識する必要がある。

まず、単数・複数・具体数 (2 つ, 3 つ……) という数性を標識するための *Enum* とサブクラスを定義する。

```

1 Enum GrammaticalNumber:
2     SINGULAR // exactly one
3     PLURAL // more than one
4
5 Class NumberSpec
6     Attributes:

```



```

7      grammatical: GrammaticalNumber | null // grammatical number
8      value: double | null // concrete cardinal value (e.g., 1, 2, 3)
9
10     Constructor(g: GrammaticalNumber | null, v: double | null):
11         grammatical = g
12         value = v

```

Listing 19 数性サブクラス定義

オブジェクトへ、これを属性として setter 関数と共に追加する。

```

1  Attributes:
2      number: NumberSpec* | null // number specification (grammatical + cardinal)
3
4  Method setNumber(spec: NumberSpec*):
5      number = spec

```

Listing 20 数性属性追加

用例を以下に示す。

```

1  // Example: "apple" (singular, unspecified cardinal)
2  N = new Lexon("apple")
3  Sing = new NumberSpec(GrammaticalNumber.SINGULAR, null)
4  N.setNumber(Sing)

```

Listing 21 単数

```

1  // Example: "two apple" (plural, value = 2)
2  N = new Lexon("apple")
3  Two = new NumberSpec(GrammaticalNumber.PLURAL, 2)
4  N.setNumber(Two)

```

Listing 22 複数

3.2.5 時間性

時間性について、例えば自然言語における過去表現の種類は、popular な単純過去・半過去・過去完了を始めとして多岐にわたり、これが逆方向—すなわち未来についても同等ぶん存在することになる。この数の語形変化のすべてを覚えるには煩雑をきわめるが、しかしながら essential に見ればこれらは、期間方向 (始点・終点)・持続性 (継続・完了)・具体時刻 (X 日, Y 時) の組み合わせに還元できる。

これらをオブジェクトの属性に加えるべく、Enum ならびにサブクラスを定義する。

```

1  // Exclusively marks whether a Lexon is temporal or spatial (or unspecified)
2  Enum SpatiotemporalKind:
3      TEMPORAL
4      SPATIAL
5
6  // Temporal aspect state
7  Enum DurationState:
8      COMPLETED // bounded/achieved state
9      ONGOING // unbounded/continuing state
10
11  // Generic directional span usable for time or space.
12  // Semantics (temporal vs spatial) are determined by the host Lexon.
13  Class Direction
14      Attributes:
15          from: Lexon* | null // origin (inclusive or context-defined)

```

```

16         to: Lexon* | null // goal (inclusive or context-defined)
17
18     Constructor(fromObj: Lexon* | null, toObj: Lexon* | null):
19         from = fromObj
20         to = toObj
21
22     Method setFrom(x: Lexon* | null): from = x
23     Method setTo(x: Lexon* | null): to = x

```

Listing 23 時間性サブクラス定義

オブジェクトへ、以下の属性と setter 関数を追加する。なお、ここで導入する *Direction* は、次に導入する属性『空間性』においても使用されるほか、「彼から私へ物を渡した (=私が彼から物を受け取った)」という譲渡や「私は彼から聞いた」という伝聞など、汎用的な『方向概念』として使用されることを想定する。

```

1  Attributes:
2      // spatiotemporal annotations
3      stKind: SpatiotemporalKind | null // TEMPORAL / SPATIAL / null
4      // temporal annotations
5      time: Lexon* | null // temporal point ("when")
6      durationState: DurationState | null // COMPLETED / ONGOING
7      direction: Direction* | null // generic span (time now; space later)
8
9      // spatiotemporal setters
10     Method setSpatiotemporalKind(k: SpatiotemporalKind): stKind = k
11     // temporal setters
12     Method setTime(x: Lexon*): time = x
13     Method setDurationState(state: DurationState): durationState = state
14     Method setDirection(dir: Direction*): direction = dir

```

Listing 24 時間性属性追加

用例を以下に示す。

```

1  // lexical items
2  S = new Lexon("I")
3  V = new Lexon("arrive")
4  Nine = new Lexon("9"); Nine.setSpatiotemporalKind(SpatiotemporalKind.TEMPORAL)
5
6  // build clause
7  S.SetDominatee(V) // SV
8  V.setDurationState(DurationState.COMPLETED) // completed event
9
10 // temporal point
11 V.setTime(Nine) // "at 9"

```

Listing 25 1 時点 (ポイント) + 完了イベント

```

1  S = new Lexon("I")
2  V = new Lexon("run")
3  T9 = new Lexon("9"); T9.setSpatiotemporalKind(SpatiotemporalKind.TEMPORAL)
4  T10 = new Lexon("10"); T10.setSpatiotemporalKind(SpatiotemporalKind.TEMPORAL)
5
6  S.SetDominatee(V) // SV
7  V.setDurationState(DurationState.ONGOING) // ongoing process
8
9  Range = new Direction(T9, T10) // 9 → 10
10 V.setDirection(Range)

```

```

1 // since 9 (open end)
2 V1 = new Lexon("work")
3 T9 = new Lexon("9"); T9.setSpatiotemporalKind(SpatiotemporalKind.TEMPORAL)
4 Since9 = new Direction(T9, null) // from 9 → open
5 V1.setDurationState(DurationState.ONGOING)
6 V1.setDirection(Since9)
7
8 // until 10 (open start)
9 V2 = new Lexon("wait")
10 T10 = new Lexon("10"); T10.setSpatiotemporalKind(SpatiotemporalKind.TEMPORAL)
11 Until10 = new Direction(null, T10) // open → to 10
12 V2.setDurationState(DurationState.ONGOING)
13 V2.setDirection(Until10)

```

Listing 27 片側開区間 (“since 9” / “until 10”)

3.2.6 空間性

任意の対象に空間性を賦与するには、その対象が置かれている位置 (place) と、それが運動・移動する場合の方向 (始点・終点)、を表現する必要がある。これを実現するべく、以下の属性を導入する。なお、時間性の導入にて追加した属性 *stKind*, *Direction* は兼用とする。

```

1 Attributes:
2   // spatial annotations
3   place: Lexon* | null // where (spatial anchor)
4   // spatiotemporal annotations
5   stKind: SpatiotemporalKind | null // TEMPORAL / SPATIAL / null
6   //
7   direction: Direction* | null // generic span (time or space)
8
9   // spatial setters
10  Method setPlace(x: Lexon* | null): place = x
11  // spatiotemporal setters
12  Method setDirection(dir: Direction*): direction = dir

```

Listing 28 空間性属性追加

用例を以下に示す。

```

1 // Example: I go from home to school.
2 S = new Lexon("I")
3 V = new Lexon("go")
4 Home = new Lexon("home"); Home.setSpatiotemporalKind(SpatiotemporalKind.SPATIAL)
5 School = new Lexon("school"); School.setSpatiotemporalKind(SpatiotemporalKind.SPATIAL)
6
7 S.SetDominatee(V) // SV
8
9 Route = new Direction(Home, School) // home → school
10 V.setDirection(Route) // spatial span on the event

```

Listing 29 方向指定運動

```

1 // Example: I sleep at home.
2 S = new Lexon("I")

```

```

3 V = new Lexon("sleep")
4 Home = new Lexon("home"); Home.setSpatiotemporalKind(SpatiotemporalKind.SPATIAL)
5
6 S.SetDominatee(V) // SV
7 V.setPlace(Home) // event location = home

```

Listing 30 定点動作

```

1 // Example: I go from home to school at 9 o'clock
2 S = new Lexon("I")
3 V = new Lexon("go")
4 Home = new Lexon("home"); Home.setSpatiotemporalKind(SpatiotemporalKind.SPATIAL)
5 School = new Lexon("school"); School.setSpatiotemporalKind(SpatiotemporalKind.SPATIAL)
6 TimeAtNine = new Lexon("9"); Nine.setSpatiotemporalKind(SpatiotemporalKind.TEMPORAL)
7
8 S.SetDominatee(V) // SV
9 Route = new Direction(Home, School) // home → school (spatial)
10 V.setDirection(Route)
11 V.setTime(TimeAtNine) // "at 9" (temporal point)

```

Listing 31 方向指定運動 (具体時刻指定)

```

1 // Example: I meet at the park at 9 o'clock.
2 S = new Lexon("I")
3 V = new Lexon("meet")
4
5 PlaceAtPark = new Lexon("park"); Park.setSpatiotemporalKind(SpatiotemporalKind.SPATIAL)
6 TimeAtNine = new Lexon("9"); Nine.setSpatiotemporalKind(SpatiotemporalKind.TEMPORAL)
7
8 S.SetDominatee(V) // SV
9
10 // spatial anchor
11 V.setPlace(PlaceAtPark)
12
13 // temporal point
14 V.setTime(TimeAtNine)

```

Listing 32 定点動作 (具体時刻指定)

3.2.7 頻度

もう一つの程度の定量化として、頻度の定量化を可能にする。自由記述ないし単位付き値による標識を可能とする属性を、setter 関数と共にオブジェクトへ追加する。

```

1 Attributes:
2   frequency: Lexon* | ValueWithUnit | null // frequency as lexical label or measured rate
3
4 Method setFrequency(x: Lexon* | ValueWithUnit | null):
5   frequency = x

```

Listing 33 頻度属性追加

用例を以下に示す。

```

1 // Example: "I run daily"
2 S = new Lexon("I")
3 V = new Lexon("run")
4 Freq = new Lexon("daily")

```

```

5
6 S.SetDominatee(V) // SV
7 V.setFrequency(Freq) // frequency = daily

```

Listing 34 頻度表現

```

1 // Example: Measured rate — "I train 3/day"
2 S = new Lexon("I")
3 V = new Lexon("train")
4 Rate = new ValueWithUnit(3.0, "/day") // quantitative frequency
5
6 S.SetDominatee(V)
7 V.setFrequency(Rate)

```

Listing 35 単位付き数値指定

3.2.8 強度

本構想が目指す『低コンテキスト化』は主に「オブジェクトどうしの関係の明示的な標識」によって実現されるが、さらに一步進んで、程度の定量化を可能にする部品を用意する。これは、文間の結合性をなるべく下げて疎結合にし、それぞれの文の情報量を底上げする運用を期待してのことである。

程度の定量化の一つとして、強度の定量化を可能にするための *Enum* とサブクラスを定義する。なお、今回は簡単のため任意の単位 (*unit*) を受け取る作りとしたが、SI 単位系などの *Enum* を定義するほうがより厳密な定量化となるだろう。

```

1 Enum IntensityPolarity:
2     STRONG // stronger / higher degree
3     WEAK // weaker / lower degree
4
5 Class ValueWithUnit
6     Attributes:
7         value: double // numeric magnitude
8         unit: string // unit label (e.g., "N", "m/s")
9
10    Constructor(v: double, u: string):
11        value = v
12        unit = u
13
14 Class IntensitySpec
15     Attributes:
16         polarity: IntensityPolarity | null // STRONG / WEAK (optional)
17         value: ValueWithUnit | null // measured intensity (optional)
18
19     Constructor(p: IntensityPolarity | null, val: ValueWithUnit | null):
20         polarity = p
21         value = val

```

Listing 36 強度サブクラス定義

オブジェクトへ、強度の属性ならびに setter 関数を追加する。

```

1 Attributes:
2     intensity: IntensitySpec* | null // intensity annotation
3
4 Method setIntensity(spec: IntensitySpec*):
5     intensity = spec

```

Listing 37 強度属性追加

用例を以下に示す.

```
1 // Example: Verb with measured intensity — "push (intensity = 1 N)"
2 V = new Lexon("push")
3 oneN = new ValueWithUnit(1.0, "N")
4 iSpec = new IntensitySpec(null, oneN) // treated by magnitude
5 V.setIntensity(iSpec)
```

Listing 38 具体強度による標識

```
1 // Example: Noun with qualitative intensity — "strong wind"
2 N = new Lexon("wind")
3 iSpec = new IntensitySpec(IntensityPolarity.STRONG, null) // no numeric value
4 N.setIntensity(iSpec)
```

Listing 39 相対強度による標識

```
1 // Example: Combining modifier and intensity — "strongly push (1.5 N)"
2 V = new Lexon("push")
3 Adv = new Lexon("strongly")
4 Adv.setModiffee(V) // lexical adverb modifies the verb
5
6 mag = new ValueWithUnit(1.5, "N")
7 iSpec = new IntensitySpec(IntensityPolarity.STRONG, mag)
8 V.setIntensity(iSpec)
```

Listing 40 具体強度・相対強度 並置による標識

3.2.9 道具

出来事が「どの手段・道具によって実現されたか」という情報は意味構造上において必須であり, これは言語学でいう道具役 (instrument) に相当する.

オブジェクトへ以下の属性と setter 関数を追加する.

```
1 Attributes:
2   instrument: Lexon* | null // instrument/comitative means (e.g., "with X", "by X")
3
4 Method setInstrument(x: Lexon* | null):
5   instrument = x
```

Listing 41 道具属性追加

用例を以下に示す.

```
1 // Example: "I cut with knife"
2 S = new Lexon("I")
3 V = new Lexon("cut")
4 O = new Lexon("knife")
5
6 S.SetDominatee(V) // SV
7 V.setInstrument(O) // with knife
```

Listing 42 英語 with 的用例

```

1 // Example: Non-material instrument (means/manner) — "I travel by train"
2 S = new Lexon("I")
3 V = new Lexon("travel")
4 Means = new Lexon("train")
5
6 S.SetDominatee(V)
7 V.setInstrument(Means) // by train (means of transport)

```

Listing 43 英語 by 的用例

3.2.10 格

自然言語では、動詞が要求する参加者関係 (誰が行為し、誰が影響を受け、誰が何を経験し、何に比較されるか) が文法的役割・意味役割として広く研究されているが、各言語ごとに表現形式が異なり、省略・語順・格変化・前置詞などの構造に強く依存する。

Lexora ではこうした表面的差異を排し、事態の構造を安定的に表すために、ACTOR(行為者・原因者)、UNDERGOER(被影響者・目標)、EXPERIENCER(経験主体)、STIMULUS(心理述語の誘因)、RECIPIENT(受益者・受取先)、CONTENT(伝達・思考の命題内容)、COMITATIVE(共参加者)、POSSESSOR(所有者)、STANDARD(比較標準)、DOMAIN(比較の領域) といった、自然言語学のエージェント・患者・テーマ・経験者・刺激・受取手・共同行為者・所有者・比較項などに対応する普遍的役割をあらかじめ分離して、次の Enum を定義する。

これらの役割を Lexon で明示的にタグ化することで、語順や格標識に依存しない意味構造の一貫性を保ち、書記モードに最適化された「低コンテキスト・高精度」の表現を実現し、自然言語間の比較・変換・機械処理を安定させることができる。

まず Enum とサブクラスを定義し、

```

1 Enum RoleType:
2     ACTOR // doer/initiator, merges Agent and Causer
3     UNDERGOER // affected participant, merges Patient and Theme
4     EXPERIENCER // sentient that experiences a state/event (psych predicates)
5     STIMULUS // entity that triggers the experience/perception
6     RECIPIENT // endpoint of transfer (includes Beneficiary as subtype)
7     CONTENT // propositional/content complement (said/thought/caused)
8     COMITATIVE // co-participant "with" (companion/associate)
9     POSSESSOR // possessor/owner relation
10    STANDARD // comparison standard ("than X", "to X")
11    DOMAIN // comparison set/domain ("in/among Y")
12
13 Class RoleAssignment
14     Attributes:
15         role: RoleType
16         filler: Lexon* // the participant filling the role
17
18     Constructor(r: RoleType, x: Lexon*):
19         role = r
20         filler = x

```

Listing 44 格属性追加

属性と add 関数をオブジェクトへ追加する。

```

1 Attributes:
2     roles: RoleAssignment[] // generic participant roles for this predicate/node
3

```

```

4 Method addRole(r: RoleType, x: Lexon*):
5     roles.append(new RoleAssignment(r, x))

```

Listing 45 格属性追加

用例を以下に示す．これまで，SV 形は $S.SetDominatee(V)$ で表現してきたが，本属性に対応する Role がある場合は，より正確に，柔軟に表現することができるようになる．

```

1 // Example: John give Mary book
2 V = new Lexon("give")
3 John = new Lexon("John")
4 Mary = new Lexon("Mary")
5 Book = new Lexon("book")
6
7 V.addRole(RoleType.ACTOR, John) // John = agent/actor
8 V.addRole(RoleType.RECIPIENT, Mary) // Mary = recipient
9 V.addRole(RoleType.UNDERGOER, Book) // book = transferred thing

```

Listing 46 三項述語 (授与)

```

1 // Example: I fear snake
2 V = new Lexon("fear")
3 I = new Lexon("I")
4 Snake = new Lexon("snake")
5
6 V.addRole(RoleType.EXPERIENCER, I)
7 V.addRole(RoleType.STIMULUS, Snake)

```

Listing 47 心理述語

```

1 // Example: I make he run
2 Make = new Lexon("make")
3 I = new Lexon("I")
4 HeRun = new Lexon(null) // embedded clause node
5 He = new Lexon("he")
6 Run = new Lexon("run")
7 He.SetDominatee(Run)
8 HeRun.SetDominatee(He)
9
10 Make.addRole(RoleType.ACTOR, I) // causer
11 Make.addRole(RoleType.CONTENT, HeRun) // caused proposition/content
12 Make.setCausativeVoice(true)

```

Listing 48 使役 (既存の *causativeVoice* と併用)

```

1 // Example: He is made to run by teacher
2 Make = new Lexon("make")
3 He = new Lexon("he")
4 Emb = new Lexon(null)
5 Emb.SetDominatee(new Lexon("he")).SetDominatee(new Lexon("run")) // schematic
6
7 Make.addRole(RoleType.UNDERGOER, He) // promoted patient (causee)
8 Make.addRole(RoleType.CONTENT, Emb) // caused content
9 Make.setInstrument(new Lexon("teacher")) // causer via "by"
10 C = new Lexon(null); C.SetDominatee(Make)
11 C.setPassiveVoice(true); C.setCausativeVoice(true)

```

Listing 49 受動使役+道具


```

1 // Example: She said "I go"
2 Say = new Lexon("say")
3 She = new Lexon("she")
4 Quote = new Lexon(null) // quoted clause top
5 I = new Lexon("I"); Go = new Lexon("go"); I.SetDominatee(Go); Quote.SetDominatee(I)
6
7 Say.addRole(RoleType.ACTOR, She)
8 Say.addRole(RoleType.CONTENT, Quote)

```

Listing 50 伝達内容

→

```

1 // Example: John' s book
2 Book = new Lexon("book")
3 John = new Lexon("John")
4
5 Book.addRole(RoleType.POSSESSOR, John)

```

Listing 51 所有

```

1 // Example: I go with friend
2 Go = new Lexon("go")
3 I = new Lexon("I")
4 Friend = new Lexon("friend")
5
6 Go.addRole(RoleType.ACTOR, I)
7 Go.addRole(RoleType.COMITATIVE, Friend) // with friend

```

Listing 52 同伴

3.3 自然言語標準文法属性

本節では、各自然言語との接合を意識した属性を実装する．多くの自然言語において採用されている文法範疇を取り込む，という位置づけである．

3.3.1 否定

発話や記述では、出来事が「起こらなかった」「存在しない」「成立しない」といった非成立の事実を明示する必要があり、これは肯定的叙述では代替できない．こうした否定的事態を一義的に示すために否定極性 (negative polarity) を属性として導入する．

```

1 Attributes:
2     negativePolarity: boolean = false // negation flag; false = affirmative
3
4 Method setNegativePolarity(flag: boolean):
5     negativePolarity = flag

```

Listing 53 否定極性属性追加

用例を以下に示す．

```

1 // Example: Clause-level negation — "I not go"
2 // Negation scope = whole clause (set on top abstract node)
3 S = new Lexon("I")
4 V = new Lexon("go")
5 S.SetDominatee(V)

```

```

6
7 C = new Lexon(null) // clause node
8 C.SetDominatee(S)
9 C.setNegativePolarity(true) // clause is negative

```

Listing 54 節レベル否定

```

1 // Example: Predicate/VP negation — "I [not eat quickly]"
2 // Negation scope = VP only
3 S = new Lexon("I")
4 V = new Lexon("eat")
5 Adv = new Lexon("quickly")
6 Adv.setModifiee(V)
7
8 VP = new Lexon(null) // VP node
9 VP.SetDominatee(V)
10 VP.setNegativePolarity(true) // negate the VP
11
12 S.SetDominatee(VP)

```

Listing 55 動詞句否定

```

1 // Example: Default affirmative (no call, or set false)
2 // "I run" — remains affirmative because negativePolarity defaults to false
3 S = new Lexon("I")
4 V = new Lexon("run")
5 S.SetDominatee(V) // no negation set → affirmative

```

Listing 56 肯定形

3.3.2 受動態

発話者が「出来事そのもの」や「影響を受けた側」を前面に出したい場合があり、行為者 (actor) を強調したくない、あるいは不明・無関係としたいときに有効な構文に受動態がある。特に被影響者 (undergoer) に焦点を置いた記述は、事件報告・科学記述・説明文など多くの場面で自然に現れ、情報構造の調整手段として重要な役割を果たす。

情報構造属性における話題 (topic) や焦点 (focus) として標識することも可能ではあるが、多くの自然言語に存在する普遍的な言語範疇であることを踏まえて、属性として追加する。

```

1 Attributes:
2   passiveVoice: boolean = false // passive voice flag; false = active
3
4 Method setPassiveVoice(flag: boolean):
5   passiveVoice = flag

```

Listing 57 受動態属性追加

用例を以下に示す。

```

1 // Example: Clause-level passive — "window be broken (by hammer)"
2 // Set passive on the top clause node
3 S = new Lexon("window")
4 V = new Lexon("break")
5 Instr = new Lexon("hammer") // used with 'by' as an instrument/agent-like role
6
7 S.SetDominatee(V)
8 V.setInstrument(Instr)

```

```

9
10 C = new Lexon(null) // clause node
11 C.SetDominatee(S)
12 C.setPassiveVoice(true) // clause is passive

```

Listing 58 節レベル受動形

3.3.3 使役態

発話者が「ある主体が、別の主体に動作をさせる/事態を起こさせる」という因果的・能動的な関与を明示したい場面は、頻繁に存在する。単なる行為記述だけでは表せない「介入」「指示」「誘発」「強制」などの複層的な因果関係を、使役態は一つの述部構造として圧縮して表現できるため、記述の精度と焦点の操作のためにこれを属性として導入する。

```

1 Attributes:
2   causativeVoice: boolean = false // causative voice flag; false = non-causative
3
4 Method setCausativeVoice(flag: boolean):
5   causativeVoice = flag

```

Listing 59 使役態属性追加

用例を以下に示す。

```

1 // Example: Default active (non-passive, non-causative) — "he run"
2 S = new Lexon("he")
3 V = new Lexon("run")
4 S.SetDominatee(V) // SV
5 // passiveVoice = false, causativeVoice = false → active, non-causative

```

Listing 60 能動態

```

1 // Example: Active causative — "I make he run"
2 S = new Lexon("I") // causer as subject
3 V = new Lexon("make") // causative predicate
4 CompS = new Lexon("he")
5 CompV = new Lexon("run")
6 CompS.SetDominatee(CompV) // embedded SV: he → run
7
8 S.SetDominatee(V) // matrix SV: I → make
9 V.SetDominatee(CompS) // make → (he run)
10 V.setCausativeVoice(true) // causative = true (active)

```

Listing 61 能動使役態

```

1 // Example: Passive causative — "he is made to run by teacher"
2 // Both passive and causative are ON; causer expressed via instrument ("by teacher")
3 S = new Lexon("he") // causee promoted to subject
4 V = new Lexon("make") // causative predicate (passivized)
5 Causer = new Lexon("teacher") // external causer (by-phrase)
6
7 EmbS = new Lexon("he")
8 EmbV = new Lexon("run")
9 EmbS.SetDominatee(EmbV) // embedded SV: he → run
10
11 S.SetDominatee(V) // matrix SV
12 V.SetDominatee(EmbS) // make → (he run)

```

```

13 V.setInstrument(Causer) // by teacher (causer)
14 C = new Lexon(null) // top clause node
15 C.SetDominatee(S)
16 C.setPassiveVoice(true) // passive = true
17 C.setCausativeVoice(true) // causative = true

```

Listing 62 受動使役態

3.3.4 他の態

態 (voice) は、多くの自然言語が具えている基礎的な文法範疇である。以下の主要な態について、属性としてオブジェクトへ追加する。

- 中動態 (middle voice): 「主体自身の内部で自然に起こる/主体が自発的に経験する」といった、能動 (他者への働きかけ) でも受動 (外部からの一方的影響) でもない中間的・自発的な事態構造を示す
- 再帰態 (reflexive): 行為者 (actor) と被影響者 (undergoer) が同一である、すなわち主体が自分自身に作用を及ぼすという特別な事態構造を示す
- 相互態 (reciprocal): 複数の主体が互いに作用し合う双方向の対称的関係を、個々の動作として列挙せずに一つの構文で表す

```

1 Attributes:
2   middleVoice: boolean | null // middle/reflexive-like alternation (null = unspecified)
3   reflexive: boolean | null // coreference of actor and undergoer
4   reciprocal: boolean | null // mutual relation among participants
5
6 Method setMiddleVoice(flag: boolean | null):
7   middleVoice = flag
8
9 Method setReflexive(flag: boolean | null):
10  reflexive = flag
11
12 Method setReciprocal(flag: boolean | null):
13  reciprocal = flag

```

Listing 63 中動態・再帰態・相互態 属性追加

用例を以下に示す。

```

1 // Example: Middle voice — "door open (middle)"
2 Door = new Lexon("door")
3 Open = new Lexon("open")
4
5 Door.SetDominatee(Open) // SV
6 C = new Lexon(null); C.SetDominatee(Door)
7 C.setMiddleVoice(true) // middle reading (spontaneous/medio-passive)

```

Listing 64 中動態

```

1 // Example: Reflexive — "John wash himself"
2 Wash = new Lexon("wash")
3 John = new Lexon("John")
4
5 // role assignment (actor and undergoer co-refer to John)
6 Wash.addRole(RoleType.ACTOR, John)
7 Wash.addRole(RoleType.UNDERGOER, John)
8

```

```

9 C = new Lexon(null); C.SetDominatee(Wash)
10 C.setReflexive(true) // mark reflexive construction

```

Listing 65 再帰態

```

1 // Example: Reciprocal — "they love each other"
2 Love = new Lexon("love")
3 They = new Lexon("they")
4
5 // both roles refer to the same plural participant set
6 Love.addRole(RoleType.ACTOR, They)
7 Love.addRole(RoleType.UNDERGOER, They)
8
9 C = new Lexon(null); C.SetDominatee(Love)
10 C.setReciprocal(true) // mutual relation within 'they'

```

Listing 66 相互態

3.3.5 様相

発話者が、事態そのものではなく、事態に対する評価・確実性・義務・可能性などの発話者自身の態度を表す必要がある場面は頻繁に存在する。自然言語の範疇でこれは「様相 (modality)」と呼ばれ、出来事の客観的記述だけでは伝わらない「許可・能力」「必要性」「推量」「義務」「可能性」といったニュアンスを、追加の述語を用いずに一つの文構造の中へ圧縮して表現できるため、発話の意図・根拠・確信度を正確に提示するうえで有用である。

排他的に扱えるよう、様相を *Enum* として定義する。

```

1 Enum Modality:
2     CAN // ability/possibility
3     MUST // strong obligation/necessity
4     SHOULD // weak obligation/recommendation
5     POSSIBLE // epistemic possibility
6     NECESSARY // epistemic necessity

```

Listing 67 様相 Enum 定義

オブジェクトへ属性として、setter 関数と共に追加する。

```

1 Attributes:
2     modality: Modality | null // clause/phrase/lexeme-level modality
3
4 Method setModality(m: Modality):
5     modality = m

```

Listing 68 様相属性追加

用例を以下に示す。

```

1 // Example: Clause-level CAN — "I can run"
2 S = new Lexon("I")
3 V = new Lexon("run")
4 S.SetDominatee(V)
5
6 C = new Lexon(null) // clause node
7 C.SetDominatee(S)
8 C.setModality(Modality.CAN)

```

Listing 69 能力

```

1 // Example: Clause-level MUST with instrument/means — "I must go by train"
2 S = new Lexon("I")
3 V = new Lexon("go")
4 Train = new Lexon("train")
5 S.SetDominatee(V)
6 V.setInstrument(Train)
7
8 C = new Lexon(null)
9 C.SetDominatee(S)
10 C.setModality(Modality.MUST)

```

Listing 70 義務 (強制)

```

1 // Example: VP-level SHOULD — "I [should eat quickly]"
2 S = new Lexon("I")
3 V = new Lexon("eat")
4 Adv = new Lexon("quickly")
5 Adv.setModiffee(V)
6
7 VP = new Lexon(null) // VP node
8 VP.SetDominatee(V)
9 VP.setModality(Modality.SHOULD)
10
11 S.SetDominatee(VP)

```

Listing 71 義務 (推奨)

```

1 // Example: Epistemic POSSIBLE — "It [possibly rain]"
2 S = new Lexon("it")
3 V = new Lexon("rain")
4 S.SetDominatee(V)
5
6 C = new Lexon(null)
7 C.SetDominatee(S)
8 C.setModality(Modality.POSSIBLE)

```

Listing 72 可能性

```

1 // Example: Epistemic NECESSARY + negation scope — "It [necessarily not be true]"
2 S = new Lexon("it")
3 V = new Lexon("be")
4 Adj = new Lexon("true")
5 S.SetDominatee(V)
6 V.SetDominatee(Adj)
7
8 VP = new Lexon(null)
9 VP.SetDominatee(V)
10 VP.setNegativePolarity(true) // negate the VP
11 VP.setModality(Modality.NECESSARY) // modal on the same scope

```

Listing 73 必要性

3.3.6 法

自然言語では、文が「事実を述べる」のか「行為を要求する」のか「非現実的な願望を表す」のか「質問する」のか「感情的反応を示す」のかといった発話行為 (illocution) の型が、語順・語形・助動詞・イントネーションなど複数の手段によって示される。

しかしこれらの手段は言語間で大きく異なり、また省略・倒置・語用論的作用によって曖昧化することも多い。Lexora はこうした表面的な差を超えて記述の意味構造を安定させるため、IMPERATIVE(命令文)、SUBJUNCTIVE(非現実・願望・仮定の叙法)、INTERROGATIVE(疑問文)、EXCLAMATIVE(感嘆文) といった自然言語の文法範疇に対応する法 (mood) を独立の属性として導入する。

これにより、発話行為の種類を語順や語形に依存せず一義的に保持でき、内容 (述語の意味) と態度 (発話行為) を分離することで、書記ベースの低コンテキスト化と機械処理の堅牢性を実現できる。

まず、排他的にするべく *Enum* を定義し、

```
1 Enum Mood:
2     IMPERATIVE // commands/requests
3     SUBJUNCTIVE // non-real/irrealis, wishes, suggestions
4     INTERROGATIVE // questions
5     EXCLAMATIVE // exclamations
```

Listing 74 法 Enum 定義

オブジェクトへ属性として setter 関数と共に追加する。

```
1 Attributes:
2     mood: Mood | null // clause-level mood
3
4 Method setMood(m: Mood):
5     mood = m
```

Listing 75 法属性追加

用例を以下に示す。

```
1 // Example: Imperative — "Go!"
2 V = new Lexon("go")
3 C = new Lexon(null); C.SetDominatee(V) // subject omitted
4 C.setMood(Mood.IMPERATIVE)
```

Listing 76 命令

```
1 // Example: Interrogative — "Where Do you run?"
2 Where = new Lexon
3 S = new Lexon("you")
4 V = new Lexon("run")
5 S.SetDominatee(V)
6 C = new Lexon(null); C.SetDominatee(S)
7 C.setMood(Mood.INTERROGATIVE)
8
9 // null indicates a form of absence (inference), and thus functions as the target of
   interrogation
10 V.addRole(RoleType.DOMAIN, new Lexon(null))
```

Listing 77 疑問 (wh-句的表現)

```
1 // Example: Superlative: "tall-est in class"
2 Adj = new Lexon("tall")
3 Adj.setDegreeKind(DegreeKind.SUPERLATIVE)
4 Class = new Lexon("class")
5 AdjHost = new Lexon(null)
6 AdjHost.SetDominatee(Adj)
7 AdjHost.addRole(RoleType.DOMAIN, Class) // in class
```

Listing 78 最上級

```

1 // Example: Subjunctive in complement — "I suggest [he go]"
2 S = new Lexon("I")
3 V = new Lexon("suggest")
4 S.SetDominatee(V)
5
6 He = new Lexon("he")
7 Go = new Lexon("go")
8 He.SetDominatee(Go)
9 Sub = new Lexon(null); Sub.SetDominatee(He)
10 Sub.setMood(Mood.SUBJUNCTIVE)
11
12 V.SetDominatee(Sub) // suggest → (he go)

```

Listing 79 願望

```

1 // Example: Indicative (implicit) — "I eat"
2 S = new Lexon("I")
3 V = new Lexon("eat")
4 S.SetDominatee(V)
5
6 C = new Lexon(null); C.SetDominatee(S)
7 // C.mood remains null → indicative

```

Listing 80 平叙文 (無標識)

3.3.7 量化

自然言語では、「何らかの存在を述べる」のか (some/any), 「全ての対象について成り立つ」のか (every/all/each), あるいは「大多数・少数・一定割合」など集合の部分量を述べるのか (most/many/few など) といった量化の違いが, 冠詞・量化詞・語順など言語固有の手段によって表される。

しかしこれらの形式は言語ごとに大きく異なり, 省略・暗示・形態変化などの影響を受けて量化の意味が曖昧化しやすい。そこで Lexora では, 自然言語における存在量化 (∃), 全称量化 (∀), 比率的量化 (大半・多数・少数など) に対応する EXISTENTIAL・UNIVERSAL・PROPORTION を QuantType として明示的に導入する。

排他的に Enum を定義し,

```

1 Enum QuantType:
2     EXISTENTIAL // some/any/a... (∃)
3     UNIVERSAL // every/all/each (∀)
4     PROPORTION // most/many/few/percentages

```

Listing 81 量化 Enum 定義

オブジェクトへ属性として setter 関数と共に追加する。

```

1 Attributes:
2     quantType: QuantType | null // existential/universal/proportion
3
4 Method setQuantType(q: QuantType):
5     quantType = q

```

Listing 82 量化属性追加

以下に用例を示す。

```

1 // Example: Existential NP — "some apple"
2 N = new Lexon("apple")

```



```
3 N.setQuantType(QuantType.EXISTENTIAL) // ∃ apple
```

Listing 83 存在量化

```
1 // Example: Universal NP — "every student"
2 N = new Lexon("student")
3 N.setQuantType(QuantType.UNIVERSAL) // ∀ student
```

Listing 84 全称量化

```
1 // Example: Proportional (lexical) — "most people"
2 N = new Lexon("people")
3 N.setQuantType(QuantType.PROPORTION) // proportional force via lexical 'most'
```

Listing 85 比率的量化 (言語的)

```
1 // Example: Proportional (numeric) — "70% voter"
2 N = new Lexon("voter")
3 N.setQuantType(QuantType.PROPORTION)
4 Pct = new ValueWithUnit(70.0, "%") // optional numeric proportion
5 // (Optionally attach as a modifier or intensity depending on your policy)
6 Pct.setModifiee(N)
```

Listing 86 比率的量化 (数值的)

3.3.8 定性

将来的に、文脈 (context) 概念を導入し、対象化可能な一連の会話・記述において登場した行為者 (actor) ならびに被対象者 (object) のオブジェクトを一覧として保持する予定であるが、本稿ではその前段階として、自然言語の文法範疇における冠詞に相当する属性を導入する。

自然言語では、名詞句が文脈上「特定の一つ」を指すのか (the)、新たに導入される未定義の対象なのか (a/an/some)、あるいは種類一般を表すのか (generic dogs, the tiger as a species) といった指示の様態が、定冠詞・不定冠詞・無冠詞・語順・文脈など複雑な手段で表されるが、これらの手段は言語間で大きく異なり、しばしば曖昧性を伴う。

そのため Lexora では、自然言語の「定性 (definiteness)」に対応する定冠性 (definite)・不定冠性 (indefinite)・総称解釈 (generic) を独立の属性として用意し、語形に依存せずに名詞句が果たす指示機能を明確に符号化できるようにする。

排他的に *Enum* として定性 (definiteness) を定義し、

```
1 Enum Definiteness:
2     DEFINITE // contextually unique/identifiable (the)
3     INDEFINITE // non-unique/introduction (a/an/some)
4     GENERIC // kind-level/generic reference
```

Listing 87 定性 Enum 定義

オブジェクトへ属性として setter 関数と共に追加する。

```
1 Attributes:
2     definiteness: Definiteness | null
3     specificity: boolean | null
4
5 Method setDefiniteness(d: Definiteness):
6     definiteness = d
```

Listing 88 定性属性追加

用例を以下に示す.

```
1 // Example: Indefinite — "I want book (any)"
2 N = new Lexon("book")
3 N.setDefiniteness(Definiteness.INDEFINITE)
```

Listing 89 定冠性

```
1 // Example: Definite — "open window (the one in context)"
2 N = new Lexon("window")
3 N.setDefiniteness(Definiteness.DEFINITE) // uniquely retrievable from discourse/situation
```

Listing 90 不定冠性

```
1 // Example: Generic — "cat like fish (as a kind)"
2 N = new Lexon("cat")
3 N.setDefiniteness(Definiteness.GENERIC)
```

Listing 91 種

3.3.9 相対程度

物理量・能力・性質・程度といった連続的な尺度は、コミュニケーションの中で「どちらがより〜か」「どれが最も〜か」「同程度である」という関係性そのものに意味があり、それを句や別構文で逐一説明すると冗長になるため、自然言語は比較級・最上級・等級といった差異の関係を圧縮して示す文法形式を発達させた。こうした形式的な区別は、評価・選択・判断・描写といった多くの言語行為を簡潔にし、発話者の意図する比較関係を明確かつ一義的に伝達するために有用であるため、オブジェクトへ属性として導入する。

排他的に *Enum* を定義し、

```
1 Enum DegreeKind:
2     COMPARATIVE // -er, more X
3     SUPERLATIVE // -est, most X
4     EQUAL // as X (as), equal degree
```

Listing 92 Enum 定義

オブジェクトへ属性として setter 関数と共に追加する。

```
1 Attributes:
2     degreeKind: DegreeKind | null // comparative/superlative/equal
3
4 Method setDegreeKind(v: DegreeKind):
5     degreeKind = v
```

Listing 93 相対強度属性追加

用例を以下に示す.

```
1 // Example: Comparative: "tall-er than John"
2 Adj = new Lexon("tall")
3 Adj.setDegreeKind(DegreeKind.COMPARATIVE)
4 John = new Lexon("John")
5 AdjHost = new Lexon(null) // phrase node hosting the adjective if needed
6 AdjHost.SetDominatee(Adj)
7 AdjHost.addRole(RoleType.STANDARD, John) // than John
```

Listing 94 比較級

```

1 // Example: Superlative: "tall-est in class"
2 Adj = new Lexon("tall")
3 Adj.setDegreeKind(DegreeKind.SUPERLATIVE)
4 Class = new Lexon("class")
5 AdjHost = new Lexon(null)
6 AdjHost.SetDominatee(Adj)
7 AdjHost.addRole(RoleType.DOMAIN, Class) // in class

```

Listing 95 最上級

```

1 // Example: Equality: "as strong as steel" → equal to steel
2 Adj = new Lexon("strong")
3 Adj.setDegreeKind(DegreeKind.EQUAL)
4 Steel = new Lexon("steel")
5 AdjHost = new Lexon(null)
6 AdjHost.SetDominatee(Adj)
7 AdjHost.addRole(RoleType.STANDARD, Steel) // equal to steel

```

Listing 96 等級

```

1 // Example: Verb/adverb comparative: "run faster than Tom"
2 V = new Lexon("run")
3 Adv = new Lexon("fast")
4 Adv.setModiffee(V)
5 Adv.setDegreeKind(DegreeKind.COMPARATIVE)
6 Tom = new Lexon("Tom")
7 VP = new Lexon(null)
8 VP.SetDominatee(V)
9 VP.addRole(RoleType.STANDARD, Tom) // than Tom (standard for the comparison)

```

Listing 97 比較 (動詞)

3.3.10 情報源

発話に含まれる内容について、それを発話者が「どのように知ったのか」を示すことは、情報の確度を定量化するために非常に重要な情報である。ここでは、推論 (INFERENCE)・想定 (ASSUMPTION)・伝聞 (HEARSAY)・直接引用 (QUOTATIVE) を *Enum* として定義する。

```

1 Enum EvidentialityType:
2     INFERENCE // concluded from indirect signs
3     ASSUMPTION // speaker's assumption
4     HEARSAY // reported by others
5     QUOTATIVE // directly quoted source

```

Listing 98 情報源 Enum 定義

オブジェクトへ、属性ならびに setter 関数を追加する。

```

1 Attributes:
2     evidentiality: EvidentialityType | null // clause/phrase-level evidentiality
3
4 Method setEvidentiality(v: EvidentialityType):
5     evidentiality = v

```

Listing 99 情報源属性追加

用例を以下に示す。

```

1 // Example: INFERENCE clause — "[INFERENCE he be home] (evidence: footprints)"
2 S = new Lexon("he")
3 V = new Lexon("be")
4 Home = new Lexon("home")
5 S.SetDominatee(V)
6 V.SetDominatee(Home) // copular: be → home
7
8 C = new Lexon(null) // clause node
9 C.SetDominatee(S)
10 C.setEvidentiality(EvidentialityType.INFERENCE)
11
12 // optional: attach an evidence mention as a modifier of the clause node
13 Footprints = new Lexon("footprints")
14 Footprints.setModiffee(C) // evidence linked via modifier

```

Listing 100 推論

```

1 // Example: ASSUMPTION phrase — "[ASSUMPTION probably go quickly]"
2 V = new Lexon("go")
3 Adv1 = new Lexon("probably")
4 Adv2 = new Lexon("quickly")
5 Adv1.setModiffee(V)
6 Adv2.setModiffee(V)
7
8 VP = new Lexon(null) // phrase node (VP)
9 VP.SetDominatee(V)
10 VP.setEvidentiality(EvidentialityType.ASSUMPTION)

```

Listing 101 想定

```

1 // Example: HEARSAY clause — "[HEARSAY he says 'it rain']"
2 subS = new Lexon("it")
3 subV = new Lexon("rain")
4 subS.SetDominatee(subV) // SV (subordinate clause)
5
6 C = new Lexon(null) // clause node
7 C.SetDominatee(subS) // attach the SV subtree
8 C.setEvidentiality(EvidentialityType.HEARSAY)
9
10 mainS = new Lexon("he")
11 mainV = new Lexon("say")
12 mainS.SetDominatee(mainV) // SV (main clause)
13
14 mainV.SetDominatee(C)

```

Listing 102 伝聞

```

1 // Example: QUOTATIVE with quoted clause — "He say '[QUOTATIVE I go]'"
2 S1 = new Lexon("he")
3 V1 = new Lexon("say")
4 S1.SetDominatee(V1)
5
6 // quoted clause
7 S2 = new Lexon("I")
8 V2 = new Lexon("go")
9 S2.SetDominatee(V2)
10

```

```

11 Quoted = new Lexon(null) // top node for quoted clause
12 Quoted.SetDominatee(S2)
13 Quoted.setEvidentiality(EvidentialityType.QUOTATIVE)
14
15 // integrate quoted clause as object of "say" (surface policy abstracted)
16 V1.SetDominatee(Quoted)

```

Listing 103 引用

3.3.11 情報構造

発話では、共有済みの情報 (topic) と新たに提示される情報 (focus) を区別し、聞き手の注意の向かう先を制御する仕組みが必要となる。これは言語範疇において情報構造 (information structure) と呼ばれ、音声言語ではアクセント・語順・繰り返しなどの音響の手がかりによって提示されるが、Lexora は属性としてこれを導入する。

まず必要な *Enum* を定義し、

```

1 Enum InformationStructure:
2     TOPIC // what the clause is about (given/anchoring)
3     FOCUS // highlighted/new/contrastive content

```

Listing 104 情報構造 Enum 定義

これを属性として、setter 関数と共にオブジェクトへ追加する。

```

1 Attributes:
2     infoStructure: InformationStructure | null // topic/focus annotation
3
4 Method setInformationStructure(v: InformationStructure):
5     infoStructure = v

```

Listing 105 情報構造属性追加

用例を以下に示す。

```

1 // Example: Topic on a noun — "As for apple, I eat"
2 S = new Lexon("I")
3 V = new Lexon("eat")
4 O = new Lexon("apple")
5
6 // clause skeleton
7 S.SetDominatee(V) // SV
8 V.SetDominatee(O) // VO
9
10 // mark topic on the noun
11 O.setInformationStructure(InformationStructure.TOPIC)

```

Listing 106 話題 (topic)

```

1 // Example: Focus on the object — "I ate CAKE (not bread)"
2 S = new Lexon("I")
3 V = new Lexon("eat")
4 O = new Lexon("cake")
5
6 S.SetDominatee(V)
7 V.SetDominatee(O)
8
9 // mark focus on the contrasted object
10 O.setInformationStructure(InformationStructure.FOCUS)

```

3.3.12 敬語

発話内容そのものではなく、社会的関係・相互の距離感・立場調整といった対人関係のメタ情報を同時に伝えるために必要とされる。これらの形式を用いることで、話し手は「相手への配慮」「自分の位置づけ」「他者の尊重」「状況に応じた場面選択」といった社会的意味を文の外に補足するのではなく、文法的に一体化した形で表現でき、コミュニケーションの円滑性と誤解回避に寄与する。

相手への距離を詰めずに、形式的な丁寧さを付与する「丁寧表現 (polite)」, 話題 (topic) となる対象を高めるための「尊敬表現 (honorific)」, 話者または話者側の対象を低めて相手を立てるための「謙譲表現 (humble)」を、排他的に *Enum* として定義し、

```
1 Enum Register:
2     POLITE // polite/formal addressee honorific
3     HONORIFIC // subject honorification
4     HUMBLE // speaker humbling (humilific)
```

Listing 108 敬語 Enum 定義

オブジェクトへ属性として、setter 関数と共に追加する。

```
1 Attributes:
2     register: Register | null // register annotation, null=>plain
3
4 Method setRegister(r: Register):
5     register = r
```

Listing 109 敬語属性追加

用例を以下に示す。

```
1 // Example: Clause-level polite style — "I go (polite)"
2 S = new Lexon("I")
3 V = new Lexon("go")
4 S.SetDominatee(V)
5
6 C = new Lexon(null) // clause node
7 C.SetDominatee(S)
8 C.setRegister(Register.POLITE) // addressee-directed politeness
```

Listing 110 丁寧表現

```
1 // Example: Subject honorific — "Professor arrive (honorific)"
2 Prof = new Lexon("professor")
3 V = new Lexon("arrive")
4 Prof.SetDominatee(V)
5
6 C = new Lexon(null)
7 C.SetDominatee(Prof)
8 C.setRegister(Register.HONORIFIC) // honors the subject referent
```

Listing 111 尊敬表現

```
1 // Example: Speaker humble — "I humbly request"
2 S = new Lexon("I")
```

```

3 V = new Lexon("request")
4 S.SetDominatee(V)
5
6 C = new Lexon(null)
7 C.SetDominatee(S)
8 C.setRegister(Register.HUMBLE) // humbling the speaker

```

Listing 112 謙讓表現

```

1 // Example: Default/plain (no call or explicit)
2 S = new Lexon("I")
3 V = new Lexon("eat")
4 S.SetDominatee(V)
5 C = new Lexon(null); C.SetDominatee(S)
6 // C.register remains null → interpreted as plain by default policy

```

Listing 113 常体 (非敬語)

参考文献

- [1]C.F. Hockett. “The Origin of Speech”. In: *Scientific American* 203.3 (1960), pp. 88–96.
- [2]UNESCO. *Literacy*. Accessed on 2025-11-13. 2025. URL: <https://www.unesco.org/en/literacy> (visited on 2025).
- [3]G. Booij. *Inflection | The Grammar of Words: An Introduction to Linguistic Morphology*. Oxford, 2007.
- [4]Marc Brysbaert. “How many words do we read per minute? A review and meta-analysis of reading rate”. In: *Journal of Memory and Language* 109 (2019), p. 104047. issn: 0749-596X. doi: <https://doi.org/10.1016/j.jml.2019.104047>. URL: <https://www.sciencedirect.com/science/article/pii/S0749596X19300786>.
- [5]Danqi Chen and Christopher D. Manning. “A Fast and Accurate Dependency Parser using Neural Networks”. In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Reports parsing speeds over 1,000 sentences per second on an Intel Core i7 2.7 GHz CPU. Doha, Qatar: Association for Computational Linguistics, Oct. 2014, pp. 740–750. doi: 10.3115/v1/D14-1082. URL: <https://aclanthology.org/D14-1082>.
- [6]Christophe Coupé et al. “Different languages, similar encoding efficiency: Comparable information rates across the human communicative niche”. In: *Science Advances* 5.9 (2019), eaaw2594. doi: 10.1126/sciadv.aaw2594. eprint: <https://www.science.org/doi/pdf/10.1126/sciadv.aaw2594>. URL: <https://www.science.org/doi/abs/10.1126/sciadv.aaw2594>.
- [7]Slav Petrov, Dipanjan Das, and Ryan McDonald. “A Universal Part-of-Speech Tagset”. In: *Proceedings of LREC 2012*. 2012. URL: <https://aclanthology.org/L12-1115/>.
- [8]Christo Kirov et al. “UniMorph 2.0: Universal Morphology”. In: *Proceedings of LREC 2018*. Miyazaki, Japan, 2018. URL: <https://aclanthology.org/L18-1293/>.
- [9]Joakim Nivre and et al. “Universal Dependencies v2: An Evergrowing Multilingual Treebank Collection”. In: *Proceedings of LREC 2020*. 2020. URL: <https://aclanthology.org/2020.lrec-1.497/>.
- [10]Aarne Ranta. “Grammatical Framework: A Type-Theoretical Grammar Formalism”. In: *Journal of Functional Programming* 14.2 (2004), pp. 145–189. doi: 10.1017/S0956796803004738. URL: <https://www.cambridge.org/core/journals/journal-of-functional-programming/article/grammatical-framework/09C4B3F43447ADDB0F632C64F195BC9C>.

- [11]Emily M. Bender et al. “Grammar Prototyping and Testing with the LinGO Grammar Matrix Customization System”. In: *Proceedings of the ACL 2010 System Demonstrations*. Uppsala, Sweden, 2010, pp. 1–6. URL: <https://aclanthology.org/P10-4001/>.
- [12]Miriam Butt et al. “The Parallel Grammar Project”. In: *Proceedings of the LFG Conference / ACL Workshop (ParGram report)*. 2002. URL: <https://aclanthology.org/W02-1503.pdf>.
- [13]Laura Banarescu et al. “Abstract Meaning Representation for Sembanking”. In: *Proceedings of the 7th Linguistic Annotation Workshop and Interoperability with Discourse*. 2013, pp. 178–186. URL: <https://aclanthology.org/W13-2322.pdf>.
- [14]Omri Abend and Ari Rappoport. “Universal Conceptual Cognitive Annotation (UCCA)”. In: *Proceedings of the 51st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Hinrich Schuetze, Pascale Fung, and Massimo Poesio. Sofia, Bulgaria: Association for Computational Linguistics, Aug. 2013, pp. 228–238. URL: <https://aclanthology.org/P13-1023/>.
- [15]Ann Copestake et al. “Minimal Recursion Semantics: An Introduction”. In: *Research on Language and Computation* 3.2–3 (2005), pp. 281–332. URL: <https://doi.org/10.1007/s11168-006-6327-9>.